

Fake Job Posting Detection Using NLP

Mini NLP Project Report

Submitted By: **ASHIR NASAR**

Register Number: **24UBC120**

Class: **III BCA A**

Course: Natural Language Processing

1. Title

Fake Job Posting Detection Using NLP and Machine Learning

The project develops an NLP-based machine learning system to automatically identify whether an online job advertisement is **REAL / Legitimate** or **FAKE / Fraudulent**.

The system uses **TF-IDF for text feature extraction** and **Logistic Regression for classification**. It also provides a confidence score along with the final prediction.

2. Problem Statement

Online recruitment platforms contain a large number of job advertisements. Among these advertisements, some may be fraudulent and can mislead job seekers.

Fraudulent job postings may contain misleading information, unrealistic employment offers, requests for registration fees, or other suspicious characteristics. Manually checking every job advertisement is difficult when the number of postings is large.

Therefore, this project aims to develop an automated **Natural Language Processing-based system** that analyzes the textual content of a job posting and predicts whether the posting is:

- **REAL / Legitimate**

- **FAKE / Fraudulent**

The system also provides a **confidence score** for each prediction.

The project demonstrates how NLP and machine learning can be applied to improve the safety and reliability of online job-search platforms.

3. Problem Relevance

Fake job advertisements are a practical problem in online recruitment. Job seekers may encounter fraudulent advertisements while searching for employment opportunities, and identifying such advertisements manually can be time-consuming.

Natural Language Processing is suitable for this problem because job advertisements contain a significant amount of textual information. Different job postings may use different words and sentence structures, but fraudulent postings can still contain certain textual patterns that can be learned by a machine learning model.

The proposed system can be useful in:

- Online recruitment platforms
- Job-search websites
- Employment portals
- Recruitment management systems
- Job-search safety applications
- Automated job-posting screening systems

The output of the system is a predicted class, either **REAL** or **FAKE**, along with the confidence of the prediction.

4. Objectives

The main objectives of the project are:

1. To obtain a real-world fake job posting dataset from Kaggle.
2. To explore and understand the dataset.
3. To identify missing values in the dataset.
4. To analyze the distribution of legitimate and fraudulent job postings.
5. To identify important textual fields from job advertisements.
6. To combine multiple textual fields into a single text representation.
7. To preprocess the job-posting text.
8. To convert textual information into numerical features using TF-IDF.
9. To train a Logistic Regression classification model.
10. To classify job postings as REAL or FAKE.
11. To calculate the confidence of each prediction.
12. To evaluate the model using accuracy, precision, recall and F1-score.
13. To visualize classification performance using a confusion matrix.
14. To analyze the prediction confidence distribution.
15. To test the trained model using new job postings.
16. To save the trained model and TF-IDF vectorizer for future use.

5. Dataset

5.1 Dataset Name

The dataset used in this project is the **Real / Fake Job Posting Prediction dataset**.

The dataset is based on the **Employment Scam Aegean Dataset (EMSCAD)** and contains real-world job advertisements labelled according to whether they are legitimate or fraudulent.

5.2 Dataset Source

The dataset was obtained from **Kaggle** and loaded into the Google Colab environment as a CSV dataset.

The dataset contains information about job advertisements, including their textual descriptions and several additional attributes.

5.3 Dataset Size

The dataset contains:

17,880 job postings

and

18 attributes/columns.

The dataset is therefore sufficiently large to demonstrate an NLP-based binary text-classification system.

5.4 Dataset Attributes

The dataset contains the following attributes:

ATTRIBUTE	DESCRIPTION
JOB_ID	Unique identifier of the job posting
TITLE	Title of the job
LOCATION	Location of the job
DEPARTMENT	Department associated with the job
SALARY_RANGE	Salary information

COMPANY_PROFILE	Information about the company
DESCRIPTION	Description of the job
REQUIREMENTS	Required qualifications and skills
BENEFITS	Benefits provided by the employer
TELECOMMUTING	Indicates whether telecommuting is available
HAS_COMPANY_LOGO	Indicates whether a company logo is present
HAS_QUESTIONS	Indicates whether questions are included
EMPLOYMENT_TYPE	Type of employment
REQUIRED_EXPERIENCE	Required experience level
REQUIRED_EDUCATION	Required educational qualification
INDUSTRY	Industry category
FUNCTION	Job function
FRAUDULENT	Target variable indicating whether the posting is fraudulent

5.5 Target Variable

The target variable is:

fraudulent

The target values are:

VALUE	MEANING
0	REAL / Legitimate
1	FAKE / Fraudulent

Therefore, this project is a **binary classification problem**.

The model receives the textual information from a job advertisement as input and predicts one of the two classes.

5.6 Class Distribution

The dataset contains:

CLASS	NUMBER OF POSTINGS
REAL / LEGITIMATE	17,014
FAKE / FRAUDULENT	866
TOTAL	17,880

The dataset contains considerably more legitimate job postings than fraudulent job postings.

This indicates that the dataset is **imbalanced**.

The minority class, FAKE job postings, represents only a small portion of the complete dataset.

5.7 Missing Value Analysis

Missing values were analyzed before performing text preprocessing.

The major missing values identified in the dataset were:

ATTRIBUTE	MISSING VALUES	PERCENTAGE
SALARY_RANGE	15,012	83.96%
DEPARTMENT	11,547	64.58%
REQUIRED_EDUCATION	8,105	45.33%
BENEFITS	7,212	40.34%
REQUIRED_EXPERIENCE	7,050	39.43%

FUNCTION	6,455	36.10%
INDUSTRY	4,903	27.42%
EMPLOYMENT_TYPE	3,471	19.41%
COMPANY_PROFILE	3,308	18.50%
REQUIREMENTS	2,696	15.08%
LOCATION	346	1.94%
DESCRIPTION	1	0.01%

The important textual fields used for NLP were handled by replacing missing values with empty strings.

5.8 Selected Text Features

The following textual fields were selected for the NLP component:

1. **title**
2. **company_profile**
3. **description**
4. **requirements**
5. **benefits**

These fields were selected because they contain meaningful textual information about the job, company, responsibilities, qualifications and benefits.

The selected fields were combined into a single text field called **combined_text**.

6. Methodology

The complete project follows the pipeline:

Kaggle Dataset → Data Exploration → Missing Value Analysis → Text Feature Selection → Text Combination → Text Preprocessing → Train-Test Split → TF-IDF → Logistic Regression → Prediction → Confidence Calculation → Evaluation

Each stage is explained below.

6.1 Dataset Loading

The dataset was uploaded to Google Colab using the file-upload functionality.

The uploaded CSV file was loaded into a Pandas DataFrame.

The dataset was then examined to understand its structure, number of records, attributes and data types.

The dataset contained **17,880 records and 18 columns**.

6.2 Data Exploration

The first five records were displayed to understand the content and structure of the job advertisements.

Dataset information was also examined using the DataFrame information function.

This helped identify:

- Column names
- Number of records
- Data types
- Non-null values
- Missing information

6.3 Missing Value Analysis

Missing values were calculated for every attribute.

This step was particularly important for textual fields because missing values cannot be directly concatenated with other text fields.

The selected textual attributes were therefore processed by replacing missing values with empty strings.

This allowed all available textual information to be safely combined.

6.4 Class Distribution Analysis

The target variable **fraudulent** was analyzed to determine the number of legitimate and fraudulent job postings.

The analysis showed:

- 17,014 REAL postings
- 866 FAKE postings

The considerable difference between the two classes indicates class imbalance.

To address this issue during model training, the Logistic Regression classifier uses:

class_weight = "balanced"

This gives greater importance to the minority class during training.

6.5 Text Feature Selection

The following fields were selected:

Title + Company Profile + Description + Requirements + Benefits

These fields contain the most relevant textual information for determining whether a job advertisement may be fraudulent.

Non-textual attributes such as job ID and numerical indicators were not used as the primary input for the NLP classification model.

6.6 Combining Text Fields

The selected text fields were combined into a single column called:

combined_text

The combined text follows the structure:

Title + Company Profile + Description + Requirements + Benefits

Combining the fields allows the model to use information from different sections of a job advertisement rather than depending on only the job title or description.

6.7 Text Preprocessing

Raw job advertisements may contain unnecessary information such as HTML tags, URLs, special characters and inconsistent capitalization.

A preprocessing function was therefore created.

The following preprocessing operations were performed:

1. Conversion of text to lowercase.
2. Removal of HTML tags.
3. Removal of URLs.
4. Removal of special characters and numbers.
5. Removal of extra whitespace.

After preprocessing, a new column called **clean_text** was created.

The cleaned text was then used as input to the TF-IDF vectorizer.

6.8 Removal of Empty Text Records

After preprocessing, the system checked whether any records contained empty text.

The number of records before preprocessing and after removing empty text records was compared.

The analysis showed:

Records before: 17,880

Records after: 17,880

Records removed: 0

Therefore, no job postings had to be removed at this stage.

6.9 Train-Test Split

The cleaned text was assigned to the input variable **X**, while the fraudulent column was assigned to the target variable **y**.

The dataset was divided into:

- **80% Training Data**
- **20% Testing Data**

A stratified split was used to maintain the proportion of REAL and FAKE postings in both datasets.

The resulting dataset sizes were:

DATASET	NUMBER OF SAMPLES
TRAINING	14,304
TESTING	3,576
TOTAL	17,880

The training data was used to learn patterns from job advertisements, while the testing data was used to evaluate the model on unseen postings.

6.10 TF-IDF Feature Extraction

Machine learning algorithms cannot directly process raw text.

Therefore, **TF-IDF (Term Frequency-Inverse Document Frequency)** was used to convert the job-posting text into numerical feature vectors.

TF-IDF assigns numerical importance to words based on their occurrence within documents and across the dataset.

The project uses:

- **Unigrams** – individual words
- **Bigrams** – combinations of two consecutive words

For example:

Unigram:

“salary”

Bigram:

“high salary”

Using both unigrams and bigrams allows the model to learn individual words as well as short phrases.

The TF-IDF vectorizer was configured with:

- Maximum features: **100,000**
- N-gram range: **(1, 2)**
- Minimum document frequency: **2**
- Maximum document frequency: **0.95**

- Sublinear TF: **True**

The resulting feature matrices contained up to **100,000 TF-IDF features**.

6.11 Avoiding Data Leakage

The TF-IDF vectorizer was fitted **only on the training data**.

The testing data was transformed using the vocabulary learned from the training data.

This is important because fitting the vectorizer on the complete dataset could allow information from the test data to influence the training process.

Therefore, the project follows a proper machine-learning workflow and reduces the possibility of data leakage.

6.12 Logistic Regression

Logistic Regression was selected as the classification algorithm.

The project is a binary classification problem:

0 → REAL

1 → FAKE

Logistic Regression is suitable for this task because it works effectively with high-dimensional sparse features such as TF-IDF vectors.

The model was configured with:

- `max_iter = 1000`
- `class_weight = "balanced"`
- `random_state = 42`

The balanced class-weight option was used to reduce the effect of class imbalance.

6.13 Prediction

After training, the Logistic Regression model was used to classify the unseen testing data.

For every job posting, the model produces one of two predictions:

REAL

or

FAKE

The predicted values were compared with the actual target values to evaluate the performance of the model.

6.14 Model Evaluation

The model was evaluated using four major classification metrics:

Accuracy

Accuracy measures the percentage of total predictions that were correctly classified.

Precision

Precision measures how many of the job postings predicted as FAKE were actually fraudulent.

Recall

Recall measures how many of the actual fraudulent job postings were correctly detected by the model.

F1-score

F1-score provides a balance between precision and recall.

F1-score is particularly useful for this project because the dataset is imbalanced and correctly identifying fraudulent job postings is important.

6.15 Classification Report

A classification report was generated for both:

- REAL
- FAKE

The report contains:

- Precision
- Recall
- F1-score
- Support

Support represents the number of actual test samples belonging to each class.

6.16 Confusion Matrix

A confusion matrix was generated to analyze correct and incorrect predictions.

The four possible outcomes are:

True Positive

A FAKE job posting correctly classified as FAKE.

True Negative

A REAL job posting correctly classified as REAL.

False Positive

A REAL job posting incorrectly classified as FAKE.

False Negative

A FAKE job posting incorrectly classified as REAL.

False negatives are particularly important in this project because a fraudulent job posting incorrectly classified as REAL could potentially mislead a job seeker.

6.17 Prediction Confidence

The Logistic Regression model also provides class probabilities using `predict_proba()`.

For every test posting, the system calculates:

- Real Probability
- Fake Probability
- Prediction
- Confidence

The confidence score is calculated as the higher probability between the REAL and FAKE classes.

A confidence value closer to **100%** indicates a stronger model prediction, while a value closer to **50%** indicates that the model is less certain between the two classes.

The confidence score represents the model's probability estimate and should not be interpreted as absolute proof that a job posting is fraudulent.

7. Implementation

7.1 Programming Language

The project was implemented using:

Python

The implementation was performed in:

Google Colab

7.2 Libraries Used

LIBRARY	PURPOSE
PANDAS	Dataset loading, manipulation and analysis
NUMPY	Numerical operations
MATPLOTLIB	Data visualization
SEABORN	Confusion matrix visualization
SCIKIT-LEARN	TF-IDF, Logistic Regression and evaluation
RE	Text preprocessing and regular expressions
JOBLIB	Saving the trained model and vectorizer
GOOGLE COLAB FILES	Dataset upload and model download

7.3 NLP Techniques Used

The major NLP techniques used in the project are:

- Text cleaning
- Lowercase conversion
- HTML tag removal
- URL removal
- Special-character removal
- Whitespace normalization
- TF-IDF feature extraction
- Unigram and bigram representation

7.4 Machine Learning Algorithm

The machine learning algorithm used is:

Logistic Regression

It was selected because it is suitable for binary classification and works effectively with sparse TF-IDF text features.

The balanced class-weight configuration was used to handle the imbalance between legitimate and fraudulent job postings.

7.5 Implementation Steps

The implementation consists of:

1. Importing required libraries.
2. Uploading the dataset.
3. Loading the CSV file.
4. Exploring the dataset.
5. Displaying dataset information.
6. Identifying dataset columns.
7. Performing missing-value analysis.
8. Analyzing class distribution.
9. Selecting relevant textual fields.
10. Handling missing textual values.
11. Combining the selected text fields.
12. Preprocessing the text.
13. Removing empty text records.
14. Defining input and target variables.

15. Splitting the dataset into training and testing sets.
16. Creating the TF-IDF vectorizer.
17. Fitting TF-IDF on the training data.
18. Transforming the testing data.
19. Training the Logistic Regression model.
20. Generating predictions.
21. Calculating evaluation metrics.
22. Generating a classification report.
23. Generating a confusion matrix.
24. Calculating prediction probabilities.
25. Analyzing prediction confidence.
26. Visualizing predicted classes.
27. Testing new job advertisements.
28. Saving the trained model.
29. Saving the TF-IDF vectorizer.

8. Results

8.1 Dataset Analysis Results

The dataset contained **17,880 job postings**.

The class distribution was:

CLASS	NUMBER
REAL / LEGITIMATE	17,014

FAKE / FRAUDULENT | 866

The analysis confirmed that the dataset is significantly imbalanced, with legitimate job postings forming the majority class.

8.2 Train-Test Results

The dataset was divided using an 80:20 stratified split.

DATASET	SAMPLES
TRAINING DATASET	14,304
TESTING DATASET	3,576

The stratified split ensured that both the training and testing datasets maintained the original class proportions.

8.3 TF-IDF Results

TF-IDF was applied using unigrams and bigrams.

The vectorizer was configured with a maximum of **100,000 features**.

The resulting matrices were:

DATASET	TF-IDF SHAPE
TRAINING	$14,304 \times 100,000$
TESTING	$3,576 \times 100,000$

This converted the textual job advertisements into numerical representations that could be processed by the Logistic Regression classifier.

8.4 Model Performance

The Logistic Regression model was evaluated using the testing dataset.

The final performance metrics generated by the notebook are:

METRIC	SCORE
ACCURACY	0.988255
PRECISION	0.857923
RECALL	0.907514
F1-SCORE	0.882022

The evaluation uses multiple metrics rather than accuracy alone because the dataset contains considerably fewer fraudulent postings than legitimate postings.

8.5 Classification Report

A detailed classification report was generated for both REAL and FAKE classes.

The report provides:

CLASS	PRECISION	RECALL	F1-SCORE	SUPPORT
REAL	1.00	0.99	0.99	3403
FAKE	0.86	0.91	0.88	173
OVERALL	0.93	0.95	0.94	3,576

The classification report provides a more detailed understanding of the model's ability to identify both legitimate and fraudulent job postings.

8.6 Confusion Matrix

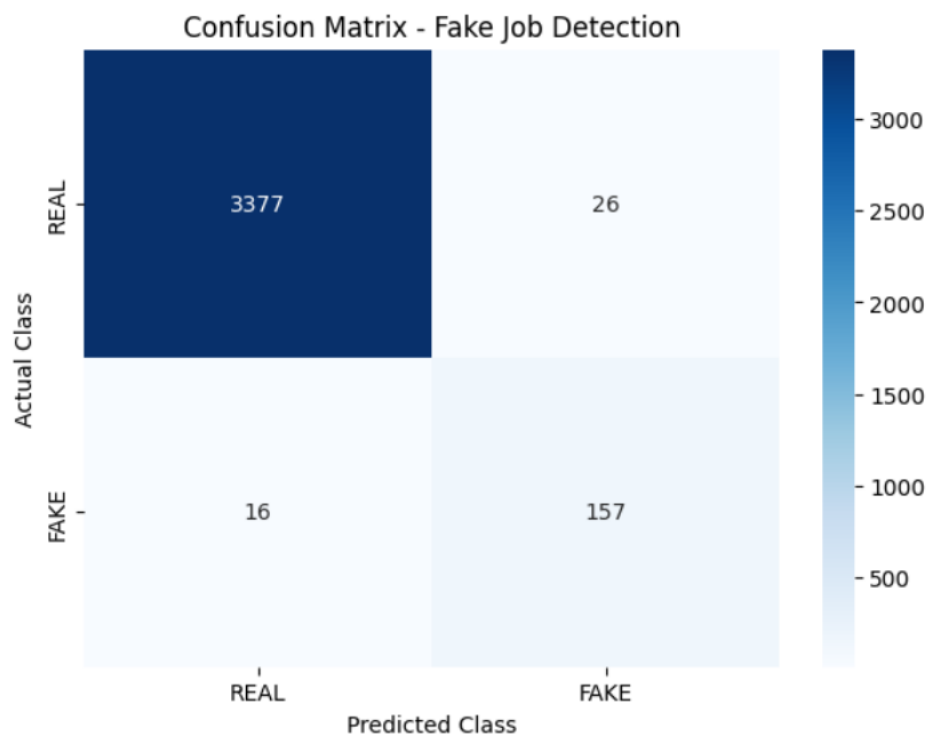
A confusion matrix was generated to visualize the classification results.

Figure 1: Confusion Matrix – Fake Job Detection

The confusion matrix shows the number of:

- REAL jobs correctly classified as REAL
- REAL jobs incorrectly classified as FAKE
- FAKE jobs correctly classified as FAKE
- FAKE jobs incorrectly classified as REAL

The confusion matrix is especially important for this project because false-negative predictions represent fraudulent job advertisements that were incorrectly classified as legitimate.



8.7 Prediction Confidence

The model also calculates the probability associated with each prediction.

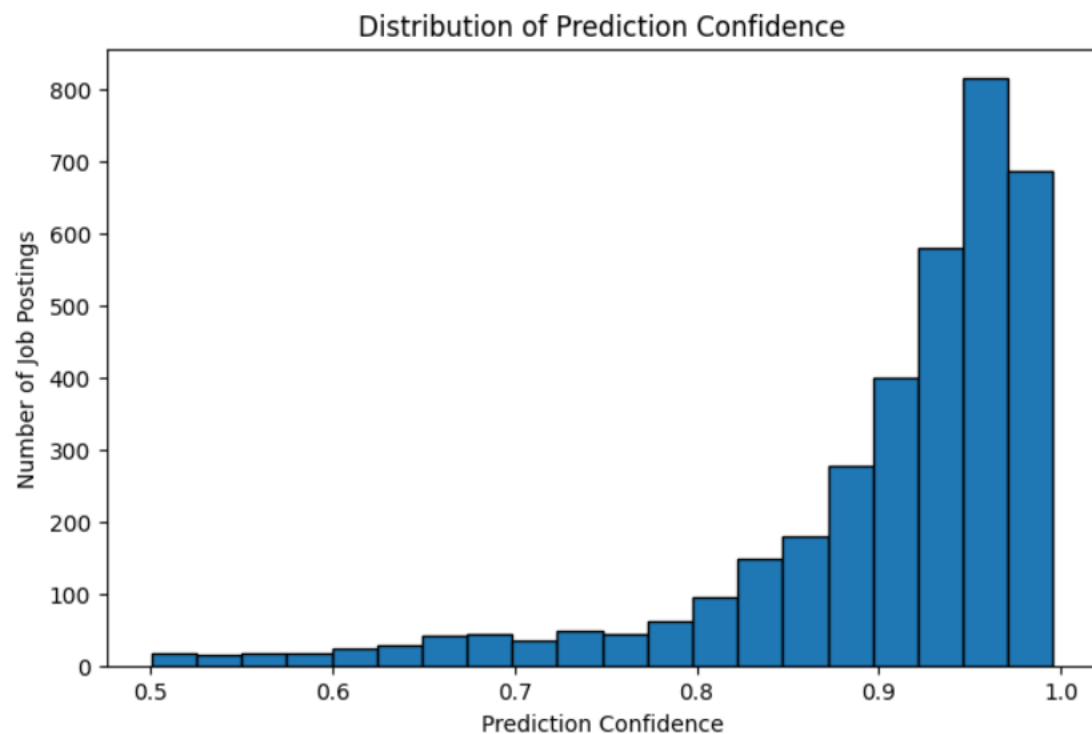
For every test job posting, the system produces:

OUTPUT	DESCRIPTION
ACTUAL	Actual class of the job posting

PREDICTED	Predicted REAL or FAKE class
REAL PROBABILITY	Probability assigned to REAL
FAKE PROBABILITY	Probability assigned to FAKE
CONFIDENCE	Probability of the predicted class

A confidence distribution graph was also generated.

Figure 2: Distribution of Prediction Confidence



A prediction closer to 100% confidence indicates that the model is more certain about its classification, while predictions closer to 50% indicate greater uncertainty.

8.8 Predicted Class Distribution

The model's predicted classes were also analyzed.

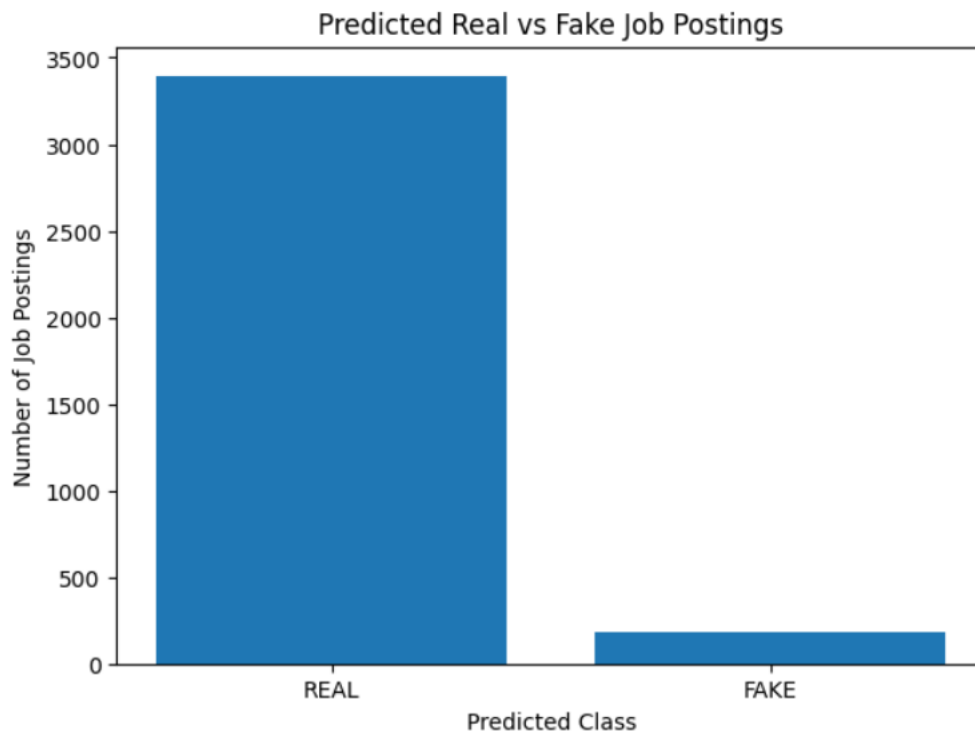
The system calculates the number of test job postings predicted as:

- REAL

- FAKE

A bar chart was generated to visualize this distribution.

Figure 3: Predicted Real vs Fake Job Postings



This provides an overview of how the trained model classifies the unseen testing dataset.

8.9 Sample Prediction – Suspicious Job Posting

The trained model was tested using a new job advertisement containing suspicious characteristics such as:

- Work-from-home claims
- Unrealistically high earnings
- No previous experience requirement
- Immediate joining
- Registration-fee request

- Request for personal information

The system processes this advertisement through:

Text Preprocessing → TF-IDF → Logistic Regression → Probability Prediction → REAL/FAKE Classification

The output contains:

- Real Probability
- Fake Probability
- Prediction
- Confidence

The prediction produced by the notebook can be inserted below:

Prediction: FAKE

Confidence: 85.26 %

8.10 Sample Prediction – Professional Job Posting

A second job advertisement describing a Software Engineer position was also tested.

The posting included requirements such as:

- Python
- SQL
- Software development experience
- Development and maintenance of software applications

The system processed this posting using the same pipeline and generated:

- Real Probability
- Fake Probability

- Prediction
- Confidence

Prediction: REAL

Confidence: 90.66 %

This demonstrates that the model can be applied to new job advertisements that were not included in the original training dataset.

8.11 Multiple Job Predictions

The system was also tested using multiple sample job advertisements.

The sample postings included:

1. A Software Engineer position.
2. A suspicious work-from-home job offering unusually high earnings.
3. A Marketing Manager position requiring professional experience.

For each posting, the system displays:

- Real Probability
- Fake Probability
- Prediction
- Confidence

This demonstrates the practical operation of the trained fake-job detection system.

8.12 Model Saving

The trained Logistic Regression model was saved as:

fake_job_model.pkl

The trained TF-IDF vectorizer was saved as:

tfidf_vectorizer.pkl

Saving these files allows the trained system to be reused later without retraining the model.

These files can also be used as the basis for deploying the model in a web application.

9. Limitations

The project has several limitations:

1. **Class imbalance:** The dataset contains significantly more legitimate job postings than fraudulent job postings.
2. **Historical dataset:** The dataset represents job advertisements from an earlier period, so modern fraudulent-job techniques may not be fully represented.
3. **Limited contextual understanding:** TF-IDF mainly captures word and phrase patterns and does not provide deep contextual understanding of the complete sentence.
4. **Incorrect classifications:** Some legitimate job postings may be incorrectly classified as fake.
5. **Missed fraudulent postings:** Some sophisticated fraudulent job advertisements may be classified as legitimate.
6. **Dependence on training data:** The model relies heavily on patterns present in the training dataset.
7. **Confidence interpretation:** The confidence score represents the model's estimated probability and should not be considered proof that a job advertisement is genuinely fraudulent.
8. **Limited NLP architecture:** The project uses traditional TF-IDF-based feature extraction rather than advanced contextual language models.

9. **Text-only focus:** The primary NLP model focuses on selected textual information and does not fully exploit all available non-textual attributes of the dataset.

10. Future Scope

The system can be improved in several ways:

1. Use **BERT or Transformer-based models** for better contextual understanding.
2. Add **multilingual fake-job detection** to support job advertisements written in different languages.
3. Include additional job-posting metadata along with textual features.
4. Develop an **explainable AI system** that identifies the words or features that influenced the model's prediction.
5. Develop a **web application** where users can paste a job advertisement and receive a REAL or FAKE prediction.
6. Develop a **browser extension** that can analyze job advertisements while users browse recruitment websites.
7. Continuously retrain the model using newer fraudulent job postings.
8. Compare Logistic Regression with other algorithms such as:
 - Support Vector Machine
 - Random Forest
 - Naive Bayes
 - Other suitable classification models
9. Use larger and more recent datasets to improve the generalization of the model.
10. Incorporate advanced fraud indicators such as suspicious URLs, contact information, salary patterns and unusual employment claims.

11. Develop a real-time job-posting screening system for recruitment platforms.
12. Integrate the model with online recruitment systems to automatically flag potentially fraudulent advertisements for further review.

11. Conclusion

This project developed an **NLP-based machine learning system for detecting fraudulent job postings**.

The project used the **Real / Fake Job Posting Prediction dataset**, containing **17,880 job advertisements**. Important textual fields such as job title, company profile, description, requirements and benefits were selected and combined into a single textual representation.

The text was then preprocessed by converting it to lowercase, removing HTML tags, URLs, special characters and unnecessary whitespace.

The cleaned text was converted into numerical features using **TF-IDF** with **unigrams and bigrams**. The dataset was divided into training and testing sets using an **80:20 stratified split**, resulting in 14,304 training samples and 3,576 testing samples.

A **Logistic Regression** classifier was trained to perform binary classification between legitimate and fraudulent job advertisements. The `class_weight="balanced"` option was used to reduce the effect of the class imbalance present in the dataset.

The model was evaluated using **accuracy, precision, recall and F1-score**, along with a classification report and confusion matrix. The system also calculates prediction probabilities and provides a confidence score for each classification.

The trained system was further tested using new job advertisements to demonstrate its practical application. The model and TF-IDF vectorizer were also saved using Joblib so that they can be reused for future applications.

Overall, the project demonstrates how **Natural Language Processing and machine learning can be applied to a practical problem in online recruitment and job-search safety**. Although the current system has limitations, particularly in contextual understanding and handling

evolving fraud patterns, it provides a useful foundation for developing a more advanced automated fake-job detection system.

Future improvements can include Transformer-based models such as BERT, multilingual support, explainable predictions, additional job metadata, continuously updated datasets and deployment through web applications or browser extensions.